

Introduction to PVM



Joint Institute for Computational Science

<http://www-jics.cs.utk.edu>
jics@cs.utk.edu

Kwai Lam Wong

Acknowledgments

Cornell Theory Center

Pittsburgh Supercomputing Center

Edinburgh Computing Center

Regional Training Center for Parallel Processing

Oak Ridge National Laboratory

Argonne National Laboratory



JICS

Joint Institute for Computational Science



Basics of Parallel Computing



Parallel Computing

- Partition a task into a set of subtasks
- Distribute subtasks to multiple processors
- Give each processor the data needed for that subtask
- Perform computations simultaneously
- Exchange data through interconnection network
- Perform some more computations
- Put the product together and output results

Issues of Parallel Computing

- Pros :
 - save elapse time
 - allow huge amount of memory access
- Cons :
 - difficult to construct
 - efficient parallel algorithm may need some thought

KEYS:

distribute subtasks optimally to balance work load
and minimize communication among processors

Parallel Programming Techniques

Data Parallelism

- each processor performs the same operation on a unique piece of data
- data mapping is critical
- domain decomposition
- programmed with HPF or message passing

Functional Parallelism

- each processor may perform a different operations
- more difficult to balance load
- commonly programmed with message passing

Message Passing Programming Paradigm

- Used primarily on distributed memory computing environment
- Since all variables are private, any data stored in remote processor's memory must be explicitly requested by programmer.
- A set of communications library is built to facilitate data transfer
- Each processor runs a sub-program written in sequential language + message communications subroutine
- More flexible, portable, however more tedious

SPMD

- Single Program, Multiple Data
- Same program runs on all processors, however, each processor operates on different set of data
- The simplest parallel parallel paradigm
- Generally contains :

```
if ( my_processor_id .eq. designated_id ) then
```

```
-----
```

```
end if
```

Message Components

- Envelope:
 - sending processor (processor_id)
 - source location (group_id, tag)
 - receiving processor (processor_id)
 - destination location (group_id, tag)
- Data (letter) :
 - data type (integer, float, complex, char....)
 - data length (buffer size, count, strides)
- Message handling

Typical Message Passing Subroutines

- Environment Identifiers
 - processor_id, group_id, initialization
- Point to Point Communications
 - blocking operations
 - non-blocking operations
- Collective Communications
 - barriers
 - broadcast
 - reduction operations

Parallel Virtual Machine

A Software Environment for Heterogeneous Distributed Computing

PVM IS

- Poor man's supercomputer :
 - Use networks of workstations as supercomputer
 - idle cycles / off hours computing resource
- Metacomputer
 - Linking multiple supercomputers
 - Very high performance
- Educational Tool
 - Easily available and simple to install
 - Simple to learn
 - Source code available for modification and examination

Why Use PVM

- Easy to learn and use
- Wide spread use
- Support from many vendors
- Simple and easily available program development, can later be moved to supercomputing platform
- Supported on many different platforms, meaning



Major Features of PVM

- Easy to install - can be done by any user, can be shared between users.
- Easy to configure virtual machine - use your own host file
- Overlapping configurations with other users
- Simple message passing interfaces
- C, C++, FORTRAN supported
- Multiple applications can run simultaneously within one virtual machine
- Package is small, only a few MB of disk space
- Highly heterogeneous - mixed networks, different machines
- Write portable code - PVM is supported on many platforms

Portability

- Ports are basically transparent to users, may be some minor differences due to machine differences
- Very portable and generic across Unix machines
- Multiprocessors - allow PVM programs to run on MPP's utilizing the vendors internal mechanisms for high speed message transfer
 - Distributed memory : T3D, Paragon, CM5, SP2
 - Shared memory : Convex/HP, SGI, Digital, Sun
- Portable to non-Unix machines
 - Windows/NT/Win95



History and Evolution

- Version 1, summer 1989, ORNL, proof of concept.
- Version 2, Feb 1991, UTK, intended as a substrate for HeNCE, first publicly released version.
- Version 3, Feb 1993, designed to be scaleable, dynamic reconfiguration, dynamic process groups, separate PVM console, improve fault tolerance, faster communication.
- Version 3.3, Shared memory, trace data with XPVM, group functions extended.
- Latest version: pvm3.3.11

Quick Overview of PVM

- PVM daemon (pvmd3)
 - A daemon program runs on each host of virtual machine
 - One per host of the virtual machine
 - Mainly a message router, provide inter-host point of contact
 - Tasks: authentication, process creation, fault detection
- Interface libraries
 - Linked with applications, calls to compose, send, receive ..
 - C programs : include pvm3.h and link to libpvm3.a
 - F programs : include fpvm3.h and link to libfpvm.a and libpvm.a
 - Group routines : link to libgpvm3.a
- PVM console : interactive control of virtual machine

Starting PVM

- Console - starts pvmd or connects to pvmd
 - `pvm [hostfile]`
- XPVM - graphical console
 - `xpvm [hostfile]`
- Manual start of daemon
 - `pvmd [hostfile]`
- Common error message
 - Can't start pvmd : check PVM_ROOT, .rhosts file
 - Can't contact local daemon : pvm crashed previously; socket file left over, remove it manually

PVM Directory

- PVM is generally installed in /usr/local/pvm3 directory
- Inset the following lines to the .cshrc file

```
#setup for pvm on UTK SP2 and Cetus Lab
setenv PVM_ROOT /usr/local/pvm3
#setenv PVM_ROOT /src/icl/pvm/pvm3
setenv PVM_ARCH RS6K
#setenv PVM_ARCH '$PVM_ROOT/lib/pvmgetarch'
set path=($PATH $HOME/pvm3/bin/$PVM_ARCH
          $PVM_ROOT/bin/$PVM_ARCH $PVM_ROOT/lib)
```
- Put all machine names in .rhosts file:

f1n1.cas.utk.edu	cetus1a
f1n2.cas.utk.edu	cetus1b
f1n3.cas.utk.edu	cetus1c
:	
- Create the pvm3/bin/RS6K (SUN4SOL2) directory in your home area

PVM Console

- Console is a task for managing the machine

%pvm

pvm> conf

1 host, 1 data format

HOST	DTSD	ARCH	SPEED
csultra1	40000	SUN4SOL2	1000

pvm>halt

%

- Adding and deleting hosts

pvm>add csultra2

1 successful

HOST	DTSD
csultra2	80000

pvm>

PVM Console (Cont'd)

pvm>conf

2 hosts, 1 data format

HOST	DTIB	ARCH	SPEED
csultra1	40000	SUN4SOL2	1000
csultra2	80000	SUN4SOL2	1000

pvm>delete csultra2

1 successful

HOST	STATUS
csultra2	deleted

pvm>quit

pvmd still running

%pvm

pvm>halt <----- *shut down pvm*

%

PVM Hostfile

- Hostfile lists machines and options for PVM
- Options at the end of each line:
 - **lo = name** **Different login name**
 - **so = pw** **Prompt for password**
 - **dx = file** **Pvmd executable**
 - **ep = path** **Task executable search path**
 - **ip = name** **Alternate IP address**
- Options at the beginning of a line :
 - **#** **Comment line**
 - ***** **Change default settings**
 - **&** **Set options only, do not start PVM on host**
- Example hostfile contents:
 machine_name1
 machine_name2.xx.xxx.edu ep = /home/username/in/myrun



Hello World : Master (hellom)

```
#include <stdio.h>
#include "pvm3.h"
#define NTASKS 3

main ()
{
    int mytid, tids[NTASKS], rc;
    printf("Enrolling master task in PVM ... \n");
    mytid = pvm_mytid();
    printf ( " I'm the master, my task id is t%x\n", mytid ) ;
    printf("Spawning worker tasks .... \n");
    rc = pvm_spawn("hellow", NULL, PvmTaskDefault, "", NTASKS, tids);
    printf( "All done. Leaving hellom.\n" ) ;
    pvm_exit () ;
}
```

Hello World : Worker (hellow)

```
#include <stdio.h>
#include "pvm3.h"

main()
{
    int mytid, ptid;
    mytid = pvm_mytid();
    ptid = pvm_parent() ;
    printf(" My worker task id is t%x\n", mytid);
    printf(" My parent id is t%x\n", ptid);
    printf(" Hello World \n");
    pvm_exit();
}
```

Compiling Hello World

- By default PVM looks for executables in
\$HOME/pvm3/bin/\$PVM_ARCH
- C :

```
cc -I$PVM_ROOT/include -o $HOME/pvm3/bin/$PVM_ARCH/hellom  
hellom.c -L/$PVM_ROOT/lib/$PVM_ARCH -lgpvm3 -lpvm3  
cc -I$PVM_ROOT/include -o $HOME/pvm3/bin/$PVM_ARCH/hellow  
hellow.c -L/$PVM_ROOT/lib/$PVM_ARCH -lgpvm3 -lpvm3
```
- FORTRAN :

```
f77 -I$PVM_ROOT/include -o $HOME/pvm3/bin/$PVM_ARCH/hellom  
hellom.f -L/$PVM_ROOT/lib/$PVM_ARCH -lfpvm3 -lgpvm3 -lpvm3  
f77 -I$PVM_ROOT/include -o $HOME/pvm3/bin/$PVM_ARCH/hellow  
hellow.f -L/$PVM_ROOT/lib/$PVM_ARCH -lfpvm3 -lgpvm3 -lpvm3
```

Makefile

```
#           Typical PVM Makefile, %make hello

CC          =  cc
MASTX       =  hellom
WORKX       =  hellow
MASTS       =  hellom.c
WORKS       =  hellow.c
INCL        =  -I${PVM_ROOT}/include
CLIBS       =  -L${PVM_ROOT}/lib/${PVM_ARCH} -lgpvm3 -lpvm3

hello : ${MASTX} ${WORKX}
    mv ${MASTX} ${WORKX} ${HOME}/pvm3/bin/${PVM_ARCH}
${MASTX} : ${MASTS}
    ${CC} ${MASTS} ${INCL} ${CLIBS} -o ${MASTX}
${WORKX} : ${WORKS}
    ${CC} ${WORKS} ${INCL} ${CLIBS} }-o ${WORKX}
```

Running Hello World

% pvm hostfile

pvm> conf

2 host , 1 data format

HOST	DTID	ARCH	SPEED
csultra1	40000	SUN4SOL2	1000
csultra2	80000	SUN4SOL2	1000

pvm> quit

pvmd still running

/%\$HOME/pvm3/bin/\$PVM_ARCH/hellom

Enrolling master task in PVM...

I am the master, my task id is t40002

Spawning worker task ...

All done. Leaving hellom

%pvm

pvmd already running.

pvm> halt

%

Output from hellow

```
%cd /tmp
```

```
%id
```

```
uid=496 (jicsx23) gid=900 (guest) groups= 900 (guest)
```

```
%cat pvml.496
```

```
[t80040000] csultra1 (204.29.64.51:1675) SUN4SOL2 3.3.11
```

```
[t80040000] ready Thu Aug 20 14:56:09 1997
```

```
[t80040000] [t40003] My worker id is t40003
```

```
[t80040000] [t40003] My parent id is t40002
```

```
[t80040000] [t40003] Hello world
```

```
[t80040000] [t80002] My worker id is t80002
```

```
[t80040000] [t80002] My parent id is t40002
```

```
[t80040000] [t80002] Hello world
```

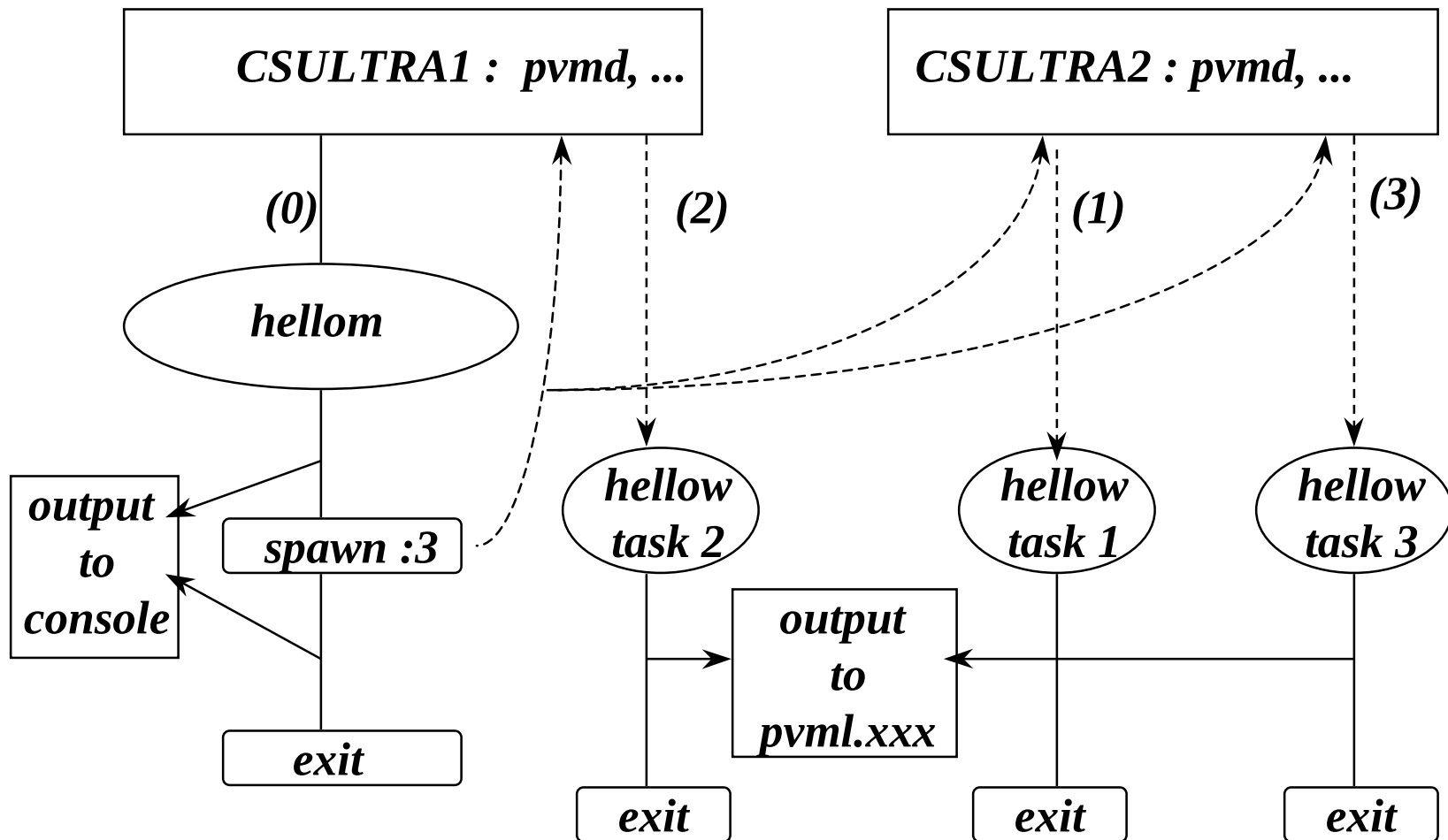
```
[t80040000] [t80001] My worker id is t80001
```

```
[t80040000] [t80001] My parent id is t40002
```

```
[t80040000] [t80001] Hello world
```



Program Flow Chart



PVMD tmp Files

- pvmd socket address file
 - pvmd publishes its socket in /tmp/pvmd.uid
 - Presence shows that pvmd is running on host
 - If pvmd is killed catastrophically, this file must be removed by hand before another pvmd will start again
- pvmd error log file
 - Errors and warnings are sent initially to stderr, then, after pvm is up and running to /tmp/pvml.uid on each host
 - Unclaimed output also goes here
 - To find your uid : use *id*



PVM Library Calls

- PVM is written in C. C++ applications can link easily. FORTRAN applications can call the routines through a FORTRAN 77 interface.
- For most PVM C functions ***pvm_routine*** there is a FORTRAN subroutine ***pvmfroutine*** with a similar interface.
- Examples :

int tid = pvm_mytid (void) ---> C

call pvmfmytid(tid) -----> FORTRAN

**int numt = pvm_spawn(char *task, char **argv, int flag, char
*where, int ntask, int *tids) -----> C**

**call pvmfspawn(taskexe, flag, where, ntask, tids, numt) -----
--> FORTRAN**

PVM Task Control

These calls allow PVM to dynamically manipulate the tasks

- **pvm_mytid**
 - Returns the task id of the PVM process, registers process in virtual machine on the first call
- **pvm_exit**
 - Notify pvmd that task exits from PVM
- **pvm_spawn(task, argv, flag, where, ntask, tids)**
 - Start ntask copies of an executable file task
- **pvm_kill (tid)**
 - Terminate another PVM task
- **pvm_pstat(tid)**
 - Query status of another PVM task
- **pvm_catchout(file)**
 - Catch output of any future child processes in the specified file

PVM Process Information

These calls provide various pieces of information about the virtual machine and the tasks

- **pvm_mytid**
- **pvm_parent**
 - Returns the TID of the process that spawned this task or **PvmNoParent** if it was not created by **pvm_spawn**
- **pvm_config(nhost, narch, dtid, name, arch, speed, info)**
 - Returns information about the configuration of the virtual machine
- **pvm_tasks (which, ntask, tid, ptid, dtid, flag, aout, info)**
 - Returns information about the PVM tasks running on the virtual machine

Message Passing Libraries

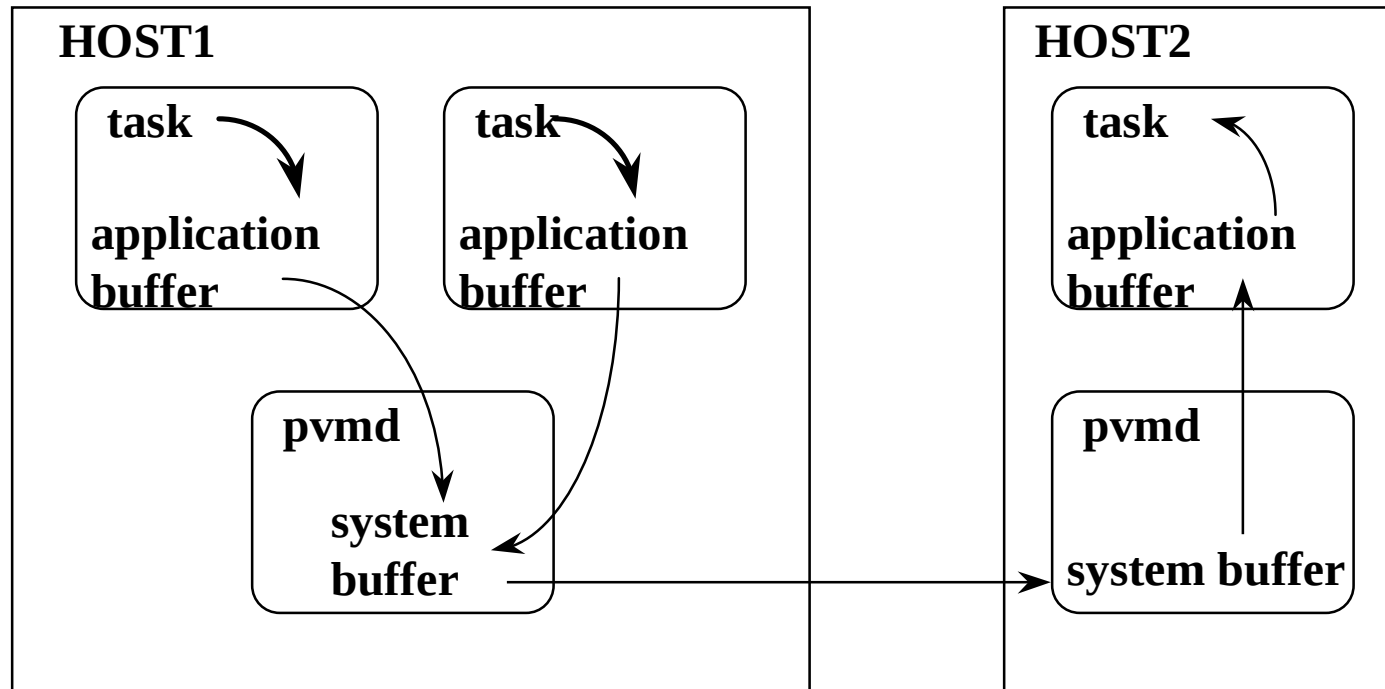
Three step sending, init, pack, send

- **pvm_initsend(encoding)**
 - Initialize send buffer, clearing current one
 - Encoding can be PvmDataDefault, PvmDataRaw, PvmDataInPlace
- **pvm_pktype(data, numitems, stride)**
 - Pack data of type into current send buffer
- **pvm_send(desttid, tag) , pvm_mcast(desttids, tag)**
 - Send of multicast the buffer to task (s) with TID (s) desttid (s) using tag to identity message, returns when it is safe to reuse the buffer.

Receive, recv, unpack

- **pvm_recv(source, tag)**
 - Receive message from source with tag, a blocking call
- **pvm_upktype(data, numitems, stride)**
 - Unpack the current buffer. Must match the packing calls.

Communication in PVM



- Default communication method
- Messages of arbitrary lengths and mixed data types, message packing allows for incompatible host formats (XDR)



Faster Communication

- For one fixed data type in contiguous form
- Get rid of buffering on MPP machines where the communication latency is relatively low
- `pvm_psend( dest, tag, data, num_items, data_type)`
 - **Pack DataInPlace and send using Unix port**
- `pvm_precv( source, tag, data, num_items, data_type, asource, atag, alength)`
- Can be as fast as native calls for multiprocessor implementations



Data Encoding

- Data encoding are associated with message buffers, e.g. `pvm_initsend( encoding)`
- `PvmDataDefault` - XDR
 - **Data translated into eXternal Data Representation, copied to message buffer, can be understood by a heterogeneous machine**
- `PvmDataRaw` - no encoding
 - **Data copied to message buffer is not encoded, it is to be received by a machine that understand the native format**
- `PvmDataInPlace` - data left in place
 - **Data not copied to the message buffer, only descriptors and pointers to the data, user must not modify data, dense data only, implies Raw, data evaluated at the send time**

Dynamic Groups

- Applications must link to the group library libgpvm3.a (-lgpvm3)
- Collective function, available via the optional group library, managed by a group server task, pvmgs.
- pvmgs the PVM group server task must be accessible (usually in \$PVM_ROOT/bin/\$PVM_ARCH)
- Tasks in group can take part in collective actions (broadcast, scatter, gather, reduce, barrier)
- Task may broadcast to groups of which it is not a member
- Groups may be frozen (made static) to improve processing, reducing communication to group server

Group Library Functions

- **pvm_joiningroup(group)**
 - Join group, where group is an arbitrary character string
- **pvm_barrier (group, count)**
 - synchronize group at the barrier
- **pvm_bcast (group, msgtag)**
 - Broadcast current message buffer to all members (except self) in the group
- **pvm_reduce(op, vp, cnt, type, msgtag, group, root)**
 - Perform global operation across the group (PvmMax, PvmMin, PvmSum, Pvmproduct), the result of the reduction operation appears on inum root
- **pvm_scatter(...) , pvm_gather(...)**
- **pvm_lvgroup (group)**
- **pvm_gettid(group, inum) , pvm_getinst(group, tid)**
 - Convert between group instance and task id TID
- **pvm_freezegroup(group, size)**
 - Makes a group static, allowing group information to be cached by members

Example : SPMD

```
c-----  
c   SPMD FORTRAN example to illustrate group function  
c-----  
  
    program fspmd  
    include 'fpvm3.h'  
    PARAMETER (NPROC = 4)  
  
    integer mytid, me, info  
    integer tids(0 : NPROC)  
  
c----Enroll in PVM  
    call pvmfmytid( mytid )  
c----Join a group and if I am the first instance, I.e. me=0 spawn more copies of myself  
    call pvmfjoininggroup( 'foo', me)  
    print*, 'me = ', me, ' mytid = ', mytid, ' number of process = ', NPROC  
    if ( me .eq. 0 ) then  
        call pvmfspawn('fspmd', PVMDEFAULT, '*', NPROC-1, tids(1), info)  
    end if  
c----wait for everyone to startup before proceeding  
    call pvmfbarrier( 'foo', NPROC, info)
```

SPMD (Cont'd)

```
c-----  
    if ( me .eq. 0) print*, 'setup complete'  
    call dowork( me, NPROC)  
c----program finished, leave group and exit pvm  
    call pvmflvgroup ( 'foo', info)  
    call pvmfexit(info)  
    stop  
    end  
  
c-----  
c  subroutine to pass a token around a ring  
c-----  
    subroutine dowork( me, nproc)  
    include 'fpvm3.h'  
    integer me, nproc  
    integer token, src, dest, count, stride, msgtag, istag  
    external PvmSum  
  
c----determine neighbor in the ring  
    if ( me .eq. 0) call pvmfgettid( 'foo', NPROC-1, src)
```

SPMD (Cont'd)

```
c-----  
if (me .eq. NPROC-1) call pvmfgettid( 'foo', 0, dest)  
if (me .gt. 0) call pvmfgettid('foo', me-1, src)  
if ( me .lt. nproc) call pvmfgettid( 'foo', me+1, dest)  
count = 1  
stride = 1  
msgtag = 4  
istag = 99  
if ( me .eq. 0) then  
    token = 0  
    call pvmfinitsend( PVMDEFAULT, info)  
    call pvmfpack( INTEGER4, token, count, stride, info)  
    call pvmfsend( dest, msgtag, info)  
    call pvmfrecv( src, msgtag, info)  
    call pvmfunpack( INTEGER4, token, count, stride, info)  
    token = token + 1  
    print*, 'token ring done'  
    print*, 'final value of token = ', token
```

SPMD (Cont'd)

```
c-----  
    else  
        call pvmfrecv( src, msgtag, info)  
        call pvmfunpack( INTEGER4, token, count, stride, info)  
        token = token +1  
        call pvmfinit send( PVMDEFAULT, info)  
        call pvmfpack( INT4, token, count, stride, info)  
        call pvmf send( dest, msgtag, info)  
    endif  
  
    call pvmf reduce( PvmSum, token, 1, INTEGER4, istag, 'foo', 0, info)  
    call pvmf barrier('foo', nproc, info)  
    if ( me .eq. 0 ) print*, ' token sum = ', token  
  
    return  
end
```



Makefile

FF = f77

EXEC = fspmd

SRC = spmd.f

INCL = -I\${PVM_ROOT}/include

FLIBS = -L\${PVM_ROOT}/lib/\${PVM_ARCH} -lfpvm3 -lgpvm3 -lpvm3

spmd : \${EXEC}

mv \${EXEC} \${HOME}/pvm3/bin/\${PVM_ARCH}

\${EXEC} : \${SRC}

\${FF} \${SRC} \${INCL} \${FLIBS} -o \${EXEC}

Running SPMD

%pvm hostfile

pvm> conf

4 hosts, 1 data format

HOST	DTID	ARCH	SPEED
r26n05	40000	RS6K	1000
r26n06	80000	RS6K	1000
r26n07	c0000	RS6K	1000
r26n08	100000	RS6K	1000

pvm>quit

pvmd still running

%make spmd

%~/pvm3/bin/RS6K/fspmd

me = 0 mytid = 262244 number of process = 4

setup complete

token ring done

final value of token = 4

token sum = 10

%

Program Description

